

Amazon Bedrock AgentCore

Strategic Assessment for MDC MCP/RAG Platform

Full Python Port vs. Container Deployment:
Capabilities, Trade-offs, and Quarterly Roadmap

Terrence McGuinness
Modeling Development Center
National Weather Service, NOAA
Terry.McGuinness@noaa.gov

April 22, 2026

Abstract

This document assesses the strategic options for integrating the MDC MCP/RAG Server—a 51-tool AI development platform for NOAA’s Global Forecast System—with Amazon Bedrock AgentCore. We evaluate two paths: (A) container deployment of the existing Node.js server to AgentCore Runtime, and (B) a full port to Python using the Strands Agents SDK with native AgentCore primitives (Memory, Gateway, Policy, Observability, Evaluations, Browser, Code Interpreter, and Registry). Path A delivers immediate production deployment with minimal effort. Path B unlocks autonomous multi-agent orchestration, persistent cross-session memory, Cedar-based policy enforcement, OpenTelemetry observability, and integration with the broader Bedrock foundation model ecosystem. We present a phased quarterly roadmap that begins with container deployment (Q3 2026) and progressively adopts AgentCore primitives, culminating in a fully Bedrock-native agentic platform (Q4 2026–Q1 2027).

Keywords: AgentCore, Bedrock, Strands Agents, MCP, Python, Multi-Agent Orchestration, Memory, Gateway, Cedar Policy, OpenTelemetry

Contents

1	Executive Summary	3
1.1	The Two Paths	3
1.2	Recommendation	3
2	Current Architecture Baseline	3
2.1	System Inventory	3
2.2	Code Volume	4
3	AgentCore Platform: 9 Services Deep Dive	4
3.1	Runtime	4
3.2	Memory	5
3.3	Gateway	5
3.4	Identity	5
3.5	Code Interpreter	5

3.6	Browser	5
3.7	Policy (Cedar)	6
3.8	Observability	6
3.9	Evaluations	6
3.10	Registry	6
4	Path A: Container Deployment (Q3 2026)	7
4.1	Scope	7
4.2	Implementation Steps	7
4.3	What Path A Delivers	7
4.4	What Path A Does NOT Deliver	7
5	Path B: Full Python Port with Strands Agents (Q4 2026–Q1 2027)	7
5.1	What a Python Port Enables	7
5.2	Strands Agents SDK Capabilities	8
5.3	Example: Autonomous Impact Analysis Agent	8
5.4	Port Strategy: Module-by-Module	9
5.5	What Already Exists in Python	9
6	New Capabilities Unlocked by Path B	9
6.1	Autonomous Multi-Agent Workflows	9
6.2	Persistent Cross-Session Memory	10
6.3	Bedrock Foundation Model Integration	10
6.4	Cedar Policy Enforcement	10
6.5	Production Observability	10
7	Risk Assessment	11
8	Quarterly Roadmap	11
9	Cost Considerations	11
10	Conclusion and Recommendation	12

1 Executive Summary

The MDC MCP/RAG Server has evolved through 52 development phases into a 51-tool platform serving AI-assisted development of the Global Forecast System. The current architecture runs on Node.js with JavaScript across 9 modules, backed by Amazon OpenSearch (vector search) and Amazon Neptune (graph queries). It works—45/45 tools pass validation, 6/6 parity benchmarks meet targets.

Amazon Bedrock AgentCore, generally available since October 2025, offers 9 modular services purpose-built for agentic AI workloads. Adopting AgentCore is not just a deployment decision—it is an architectural inflection point that determines whether the platform remains a *tool server* or evolves into an *autonomous agent ecosystem*.

1.1 The Two Paths

	Path A: Container Deploy	Path B: Full Python Port
Effort	1–2 weeks	3–4 months
Language	Keep Node.js/JavaScript	Port to Python (Strands SDK)
AgentCore Services	Runtime only	All 9 services
What It Unlocks	Managed hosting, auto-scaling, Cloud-Formation	Multi-agent orchestration, memory, policy, observability, evaluations, browser, code interpreter, registry
Risk	Low (no code changes)	Medium (full rewrite)
Strategic Value	Tactical (ship it)	Transformational

Table 1: Path comparison summary.

1.2 Recommendation

Execute both paths sequentially: Path A in Q3 2026 for immediate production deployment, then Path B as a Q4 2026–Q1 2027 initiative that progressively ports modules to Python while the Node.js server continues serving production traffic.

2 Current Architecture Baseline

2.1 System Inventory

Component	Technology	Scale
MCP Server	Node.js 20, JavaScript (ES modules)	51 tools, 9 modules
Vector Search	Amazon OpenSearch (k-NN)	85,921 docs, 768/1024-dim
Graph Database	Amazon Neptune (openCypher)	59,759 nodes, 2.6M rels
Embedding Models	MPNet-768 (local), Titan-1024, Nova (Bedrock)	6 profiles
HTTP Transport	mcp-http-server.js (Streamable HTTP)	Port 3000
SDD Framework	53 workflows, 32 sessions	JSONL persistence

Table 2: Current system inventory on `develop_aws` branch.

2.2 Code Volume

Module	Primary Files	Approx. Lines
UnifiedMCPServer	UnifiedMCPServer.js	1,300
CodeAnalysisTools	CodeAnalysisTools.js	1,300
GraphRAGTools	GraphRAGTools.js	925
OperationalTools	OperationalTools.js	894
SemanticSearchTools	SemanticSearchTools.js	~800
EE2ComplianceTools	EE2ComplianceTools.js	~600
SDDWorkflowTools	SDDWorkflowTools.js	~700
WorkflowInfoTools	WorkflowInfoTools.js	~400
NeptuneAdapter	NeptuneAdapter.js	~650
OpenSearchAdapter	OpenSearchAdapter.js	~500
GGSR Traversal	GGSRTraversalPrototypes.js	~600
GraphGuidedRetrieval	GraphGuidedRetrieval.js	~400
Ingestion Scripts (Python)	14 scripts	~4,000
Total JavaScript		~9,000
Total Python (ingestion)		~4,000

Table 3: Code volume by module. The ingestion pipeline is already Python.

3 AgentCore Platform: 9 Services Deep Dive

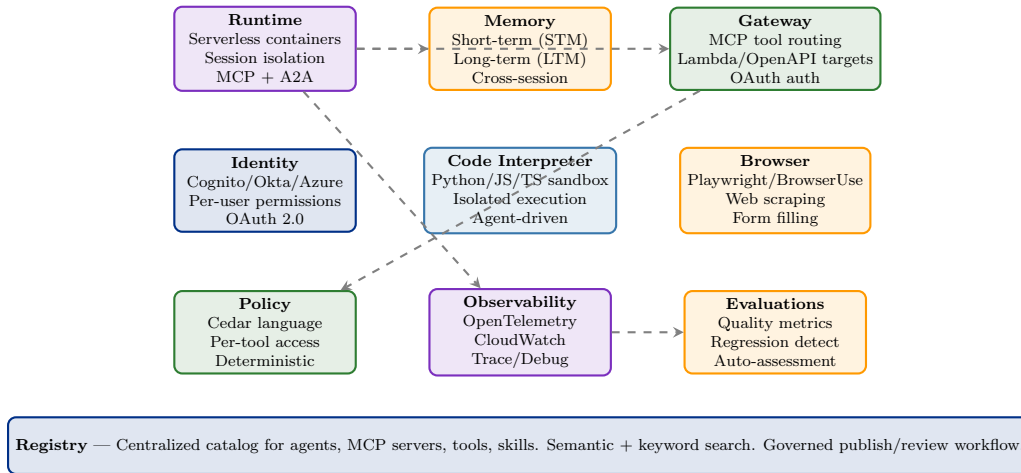


Figure 1: AgentCore’s 9 modular services. Each can be adopted independently.

3.1 Runtime

Serverless container hosting purpose-built for AI agents and MCP servers. Expects HTTP at 0.0.0.0:8000/mcp. Supports stateless and stateful Streamable HTTP. Session isolation via microVMs. Fast cold starts. CloudFormation-managed via `agentcore deploy`.

Current gap: Our server runs on a manually started process (`start-aws-mcp.sh`) with a security group hack for port 3000.

Path A value: Immediate—change port to 8000, containerize, deploy.

3.2 Memory

Managed short-term memory (conversation history within a session) and long-term memory (insights extracted across sessions). Agents can share memory stores. Integrates with Strands, LangGraph, LangChain, LlamaIndex.

Current gap: SDD session state persists to `history.jsonl` on local disk. No cross-session learning. No shared memory between agents.

Path B value: SDD sessions, query feedback, user preferences, and code analysis context could persist in AgentCore Memory, enabling agents to learn from past interactions and share knowledge.

3.3 Gateway

Converts APIs, Lambda functions, and MCP servers into unified tool endpoints. Handles OAuth authentication, routing, and rate limiting. Supports multiple targets per gateway.

Current gap: Docker MCP Gateway (legacy) or Private API Gateway (AWS) handle routing. No per-tool auth or rate limiting.

Path B value: Publish our 51 tools as Gateway targets with per-tool Cedar policies. External teams could consume specific tools (e.g., EE2 compliance checking) without access to the full server.

3.4 Identity

Agent identity management compatible with Cognito, Okta, Azure AD, Auth0. Per-user agent permissions. OAuth 2.0 flows.

Current gap: Bearer token auth (single shared token) or VPC resource policy (no per-user identity).

Path B value: Per-developer tool access. Audit trail of who used which tool. Integration with NOAA's identity provider.

3.5 Code Interpreter

Isolated sandbox for agents to execute Python, JavaScript, and TypeScript code. Agents can write and run code to solve complex tasks.

Current gap: Ingestion scripts run manually on EC2. No agent-driven code execution.

Path B value: Agents could autonomously run ingestion scripts, benchmarks, or data analysis. A Strands agent could decide “the community summaries are stale” and trigger re-ingestion without human intervention.

3.6 Browser

Cloud-based browser runtime for web interaction. Playwright and BrowserUse support. Form filling, navigation, data extraction.

Current gap: Documentation crawling uses `ingestion_base.py` with `requests` + BeautifulSoup. Rate-limited sites (MOM6, CICE) require manual retry.

Path B value: Agent-driven documentation crawling that handles JavaScript-rendered pages, CAPTCHAs, and rate limiting automatically. Could replace the entire documentation ingestion pipeline.

3.7 Policy (Cedar)

Deterministic access control using Cedar, AWS's open-source policy language. Policies are human-readable, analyzable, and schema-validated. Intercepts every tool call before execution.

Current gap: No per-tool access control. All authenticated users can call all 51 tools.

Path B value: Role-based tool access. Example Cedar policy:

```

1 permit(
2   principal is AgentCore::OAuthUser,
3   action == AgentCore::Action::"CallTool",
4   resource is AgentCore::Gateway::"mdc-mcp-rag"
5 ) when {
6   resource.toolName in [
7     "analyze_ee2_compliance",
8     "scan_repository_compliance",
9     "generate_compliance_report"
10  ] && principal.groups.contains("ee2-auditors")
11 };

```

Listing 1: Cedar policy restricting EE2 compliance tools to auditors.

3.8 Observability

OpenTelemetry-compatible tracing, debugging, and performance monitoring. Integrates with CloudWatch, Dynatrace, Elastic, and Datadog. Traces every agent step: LLM calls, tool invocations, memory retrievals.

Current gap: `health_history.jsonl` with manual drift detection. No distributed tracing. No per-tool latency breakdown.

Path B value: Full OpenTelemetry traces for every MCP tool call. Automatic anomaly detection. Integration with CloudWatch dashboards. Replaces our custom `get_health_trend` and `mcp_health_check` tools with managed infrastructure.

3.9 Evaluations

Automated agent quality assessment. Measures task execution, edge case handling, and output reliability. Integrates with CloudWatch for continuous monitoring.

Current gap: `benchmark_runner.py` with 24 ground-truth queries. Manual execution. Results stored in S3.

Path B value: Continuous automated quality monitoring in production. Regression detection on every deployment. Replaces our custom `get_quality_metrics` tool.

3.10 Registry

Centralized catalog for discovering and managing agents, MCP servers, tools, and skills. Semantic + keyword search. Governed publish/review workflow.

Current gap: Tools are discoverable only via `tools/list` MCP call. No organizational catalog.

Path B value: Publish the MDC MCP tools to an organizational registry. Other NOAA teams could discover and consume our EE2 compliance, code analysis, or operational guidance tools.

4 Path A: Container Deployment (Q3 2026)

4.1 Scope

Deploy the existing Node.js MCP server to AgentCore Runtime with zero code changes to the 51 tools. Estimated effort: 1–2 weeks.

4.2 Implementation Steps

1. Modify `mcp-http-server.js`: change port from 3000 to 8000
2. Update Dockerfile: set `EXPOSE 8000`, add `CMD ["node", "src/mcp-http-server.js", "8000", "full"]`
3. Install AgentCore CLI: `npm install -g @aws/agentcore`
4. Create project: `agentcore create --protocol MCP`
5. Configure: point entrypoint to our Dockerfile
6. Deploy: `agentcore deploy`
7. Update Kiro MCP config with AgentCore endpoint URL
8. Remove port 3000 security group rule and dev bridge

4.3 What Path A Delivers

- CloudFormation-managed deployment (IaC)
- Auto-scaling and session isolation
- OAuth authentication via Cognito
- No more manual `start-aws-mcp.sh`
- No more security group hacks

4.4 What Path A Does NOT Deliver

- No multi-agent orchestration
- No persistent memory across sessions
- No Cedar policy enforcement
- No OpenTelemetry observability
- No automated evaluations
- No agent-driven code execution or browser automation

5 Path B: Full Python Port with Strands Agents (Q4 2026–Q1 2027)

5.1 What a Python Port Enables

A full port to Python using the Strands Agents SDK transforms the platform from a passive tool server into an active agent ecosystem:

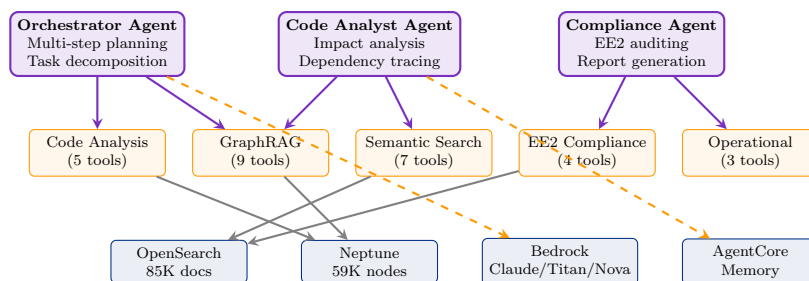


Figure 2: Path B vision: Strands agents orchestrating MCP tools with Bedrock models and AgentCore Memory.

5.2 Strands Agents SDK Capabilities

The Strands Agents SDK (open-source, Python) provides:

- **Model-driven agents:** Define agent behavior in a few lines; the LLM decides which tools to call and in what order
- **Multi-agent orchestration:** GraphBuilder for directed workflows, handoffs between specialized agents, swarm patterns
- **Tool integration:** Native MCP tool consumption—our 51 tools become first-class Strands tools
- **Memory integration:** Built-in session manager with AgentCore Memory for STM and LTM
- **Model agnostic:** Works with Bedrock (Claude, Nova, Titan), OpenAI, or any LLM provider
- **Observability:** OpenTelemetry instrumentation built-in

5.3 Example: Autonomous Impact Analysis Agent

```

1 from strands import Agent
2 from strands.tools.mcp import MCPClient
3
4 # Connect to our MCP server (all 51 tools available)
5 mcp = MCPClient("http://localhost:8000/mcp")
6
7 # Create an agent with access to our tools
8 analyst = Agent(
9     model="us.anthropic.claude-opus-4-6-v1:0",
10    tools=mcp.list_tools(),
11    system_prompt="""You are a code impact analyst for the NOAA
12    Global Forecast System. When asked about a code change:
13    1. Use get_code_context to understand the symbol
14    2. Use get_change_impact to assess blast radius
15    3. Use trace_full_execution_chain to find affected workflows
16    4. Use search_documentation for relevant docs
17    5. Produce a structured impact report."""
18 )
19
20 # The agent autonomously calls tools and reasons
21 result = analyst("Analyze the impact of changing setuprad")

```

Listing 2: Strands agent that autonomously analyzes code change impact.

5.4 Port Strategy: Module-by-Module

The port does not need to be all-or-nothing. Python and Node.js can coexist during the transition:

Phase	Module	Effort	Rationale
B1	FastMCP wrapper + tool registration	1 week	Scaffolding
B2	OpenSearch adapter (Python)	1 week	Already have <code>aws_backend.py</code>
B3	Neptune adapter (Python)	2 weeks	openCypher via <code>neo4j</code> driver
B4	SemanticSearchTools	1 week	Vector queries
B5	CodeAnalysisTools	2 weeks	Graph traversal
B6	GraphRAGTools + GGSR	3 weeks	Most complex module
B7	EE2ComplianceTools	1 week	Pattern matching
B8	OperationalTools	1 week	Hybrid queries
B9	SDDWorkflowTools	1 week	File I/O
B10	WorkflowInfoTools	3 days	Static tools
B11	Strands agent layer	2 weeks	Multi-agent orchestration
B12	AgentCore Memory integration	1 week	STM + LTM
B13	AgentCore Gateway + Policy	1 week	Cedar policies
B14	Observability + Evaluations	1 week	OpenTelemetry
Total		~16 weeks	

Table 4: Module-by-module port schedule.

5.5 What Already Exists in Python

The ingestion pipeline is already Python (~4,000 lines):

- `embedding_registry.py` — Model profiles and registry
- `embedding_provider.py` — Local and Bedrock providers
- `ingestion_base.py` — BaseIngester with backend routing
- `aws_backend.py` — OpenSearch and Neptune clients
- `benchmark_runner.py` — Quality metrics
- `drift_detector.py` — Embedding drift detection
- `fine_tuning_pipeline.py` — SageMaker training
- `hard_negative_miner.py` — Graph-powered training data
- 7 ingestion scripts (code, docs, Fortran, shell, jjobs, etc.)

These provide a foundation—the OpenSearch and Neptune client code can be reused directly in the Python MCP server.

6 New Capabilities Unlocked by Path B

6.1 Autonomous Multi-Agent Workflows

With Strands Agents, we can build specialized agents that collaborate:

Agent	Tools Used	Autonomous Behavior
Code Analyst	<code>get_code_context</code> , <code>get_change_impact</code> , <code>trace_full_execution_chain</code>	Produces impact reports for PRs without human prompting
Compliance Auditor	<code>scan_repository_compliance</code> , <code>analyze_ee2_compliance</code>	Runs nightly EE2 audits, files issues for violations
Knowledge Curator	<code>check_knowledge_integrity</code> , <code>drift_detector</code>	Detects stale embeddings, triggers re-ingestion
Documentation Crawler	AgentCore Browser	Crawls rate-limited sites with retry, handles JS rendering
Benchmark Runner	AgentCore Code Interpreter	Runs quality benchmarks on schedule, alerts on regression

Table 5: Specialized agents enabled by Strands + AgentCore.

6.2 Persistent Cross-Session Memory

AgentCore Memory enables agents to learn from past interactions:

- **Short-term:** Conversation context within a Kiro session (replaces our stateless HTTP transport)
- **Long-term:** “User frequently asks about JGLOBALFORECAST and setuprad”—agent pre-loads relevant context
- **Shared:** Multiple agents share a memory store—the Compliance Auditor’s findings are available to the Code Analyst
- **SDD sessions:** Development session state persists in managed memory instead of `history.jsonl`

6.3 Bedrock Foundation Model Integration

A Python port enables direct Bedrock API integration:

- **Claude Opus 4.6/4.7** for complex reasoning about code architecture
- **Claude Sonnet 4.6** for fast tool-calling in multi-step workflows
- **Claude Haiku 4.5** for high-volume, low-cost operations (e.g., batch compliance checking)
- **Amazon Nova** for cost-effective embedding generation
- **Model routing:** Strands can route to different models based on task complexity

6.4 Cedar Policy Enforcement

Fine-grained, deterministic access control:

- Per-tool permissions (e.g., only auditors can run compliance scans)
- Per-user rate limiting (e.g., 100 tool calls/hour for external users)
- Data-level restrictions (e.g., restrict `search_documentation` to specific collections)
- Audit trail of every policy decision

6.5 Production Observability

Replace custom health monitoring with managed infrastructure:

- OpenTelemetry traces for every tool call (latency, errors, tokens)

- CloudWatch dashboards with per-tool metrics
- Automatic anomaly detection (replaces `get_health_trend`)
- Integration with Dynatrace, Elastic, or Datadog

7 Risk Assessment

Risk	Likelihood	Impact	Mitigation
Python port introduces bugs	Medium	High	Module-by-module port with parity tests against Node.js baseline
Neptune openCypher differences resurface	Low	Medium	Already solved—port the fixes, not the bugs
AgentCore service limits	Low	Medium	Start with Runtime, add services incrementally
Strands SDK breaking changes	Low	Low	Pin SDK version, test on upgrade
Team Python proficiency	Low	Low	Ingestion pipeline already Python; team is bilingual
Port takes longer than estimated	Medium	Medium	Node.js server continues serving production during port

Table 6: Risk assessment for Path B.

8 Quarterly Roadmap

Quarter	Milestone	Deliverables
Q3 2026	Path A: Container Deploy	AgentCore Runtime deployment, Cognito auth, remove dev bridge. Node.js server in production.
Q4 2026	Path B Phase 1: Foundation	Python FastMCP server with OpenSearch + Neptune adapters. First 20 tools ported. Parity tests passing. Node.js still serving production.
Q1 2027	Path B Phase 2: Full Port	All 51 tools in Python. GGSR ported. Strands agent layer. AgentCore Memory + Gateway + Policy. Cutover from Node.js.
Q2 2027	Path B Phase 3: Agents	Multi-agent orchestration. Autonomous compliance auditor. Knowledge curator. Browser-based documentation crawler. Full observability.

Table 7: Quarterly roadmap for AgentCore adoption.

9 Cost Considerations

AgentCore uses consumption-based pricing with no upfront commitments:

- **Runtime:** Per-second billing for container execution time
- **Memory:** Per-GB-month for stored memory + per-request for retrieval
- **Gateway:** Per-request for tool invocations
- **Observability:** Standard CloudWatch pricing for traces/logs
- **Bedrock models:** Per-token pricing (varies by model)

Current cost baseline: EC2 instance (\$0.10/hr) + Neptune (\$0.35/hr) + OpenSearch (\$0.24/hr) $\approx$ \$500/month. AgentCore Runtime would replace the EC2 cost with per-invocation pricing, potentially reducing costs during low-usage periods.

10 Conclusion and Recommendation

The MDC MCP/RAG platform stands at an architectural inflection point. Path A (container deployment) is a tactical win that delivers production readiness in 1–2 weeks. Path B (full Python port) is a strategic investment that transforms the platform from a tool server into an autonomous agent ecosystem with persistent memory, policy enforcement, observability, and multi-agent orchestration.

Recommendation: Execute both paths sequentially.

1. **Q3 2026:** Deploy Node.js server to AgentCore Runtime (Path A). Ship it.
2. **Q4 2026:** Begin Python port, module by module, with parity tests against the Node.js baseline.
3. **Q1 2027:** Complete port, activate AgentCore Memory + Gateway + Policy. Cutover.
4. **Q2 2027:** Build autonomous agents. Compliance auditor. Knowledge curator. Full observability.

The Node.js server continues serving production traffic throughout the port. No downtime. No big-bang migration. Each module is validated against the existing baseline before cutover.

The question is not whether to adopt AgentCore—it is how fast to move from tactical deployment to strategic transformation.

References

1. AWS. “Amazon Bedrock AgentCore.” <https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/what-is-bedrock-agentcore.html>, 2025.
2. AWS. “Deploy MCP servers in AgentCore Runtime.” <https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/runtime-mcp.html>, 2025.
3. AWS. “Strands Agents 1.0: Production-Ready Multi-Agent Orchestration.” <https://aws.amazon.com/blogs/opensource/introducing-strands-agents-1-0-production-ready-multi-agent-orchestration/>, 2025.
4. AWS. “AgentCore Memory: Building Context-Aware Agents.” <https://aws.amazon.com/blogs/machine-learning/amazon-bedrock-agentcore-memory-building-context-aware-agents/>, 2025.
5. AWS. “Secure AI Agents with Policy in AgentCore.” <https://aws.amazon.com/blogs/machine-learning/secure-ai-agents-with-policy-in-amazon-bedrock-agentcore/>, 2025.
6. AWS. “Transform MCP Architecture: Unite MCP Servers through AgentCore Gateway.” <https://aws.amazon.com/blogs/machine-learning/transform-your-mcp-architecture-unite-mcp-servers-through-agentcore-gateway/>, 2025.
7. AWS. “Customize Agent Workflows with Advanced Orchestration using Strands Agents.” <https://aws.amazon.com/blogs/machine-learning/customize-agent-workflows-with-advanced-orchestration-using-strands-agents/>, 2025.
8. Strands Agents SDK. <https://github.com/strands-agents/sdk-python>, 2025.
9. Cedar Policy Language. <https://www.cedarpolicy.com/>, AWS, 2023.

10. Anthropic. “Model Context Protocol Specification.” <https://spec.modelcontextprotocol.io/>, 2024.